

Semantic Search Engine on MS MARCO: Comparing Word2Vec, SBERT, and Hybrid Retrieval

ATRAGADA VENKATA NAGA SAI GANESH (24110060), IIT Gandhinagar, India

MANEMONI SUMANJALI (24110195), IIT Gandhinagar, India

SIDDA DIVYA (24110337), IIT Gandhinagar, India

DIVYAM SAHU (25120028), IIT Gandhinagar, India

We have built a semantic search engine on the MS MARCO v2.1 dataset [3] and did comparison between three retrieval approaches: (1) Word2Vec with query expansion and FAISS indexing, (2) SBERT (all-MiniLM-L6-v2) [6], and (3) a Hybrid pipeline that uses Word2Vec for candidate retrieval and SBERT for reranking. We used around 1.99 million passages from 200K training samples. On 305 evaluation queries, SBERT model gave good results ($NDCG@5 = 0.438$, $Recall@5 = 0.606$, $MRR@5 = 0.386$) while Word2Vec was fastest at 933 ms. We also ran an ablation on query expansion size and found $N=3$ works better than larger values. We also built a Gradio web app to demonstrate all three designs interactively.

Additional Key Words and Phrases: Semantic Search, Word2Vec, Sentence-BERT, FAISS, Query Expansion, MS MARCO

1 Introduction

Starting with the problem with normal keyword search is that it only matches on exact words. So if you search for “car”, passages with “vehicle” will not come even though their understanding to humans is same — this is the vocabulary mismatch problem [2]. Word embeddings comes to rescue here by mapping words to vectors based on their meaning. Word2Vec [7] does this at the word level, while Sentence-BERT [6] encodes the complete sentence and understands context around it better. In this project we three designs on MS MARCO [3], used FAISS [1] for fast vector search, and then compared them on 5 IR metrics, query latency, and an ablation study on expansion size.

2 Dataset

We used the MS MARCO v2.1 dataset [4], which contains real Bing search queries with passages annotated by humans using `is_selected` flag. We have taken the first 200K training samples which gave us 1.99M passages. But during evaluation we found that 195 out of 500 queries had no relevant passage marked at all, so we skipped those and evaluated with the remaining 305 queries.

Table 1. Dataset statistics.

Field	Value
Training samples	200,000
Total passages	1,995,277
Word2Vec embeddings	1,995,141
SBERT embeddings	1,995,277
Evaluation queries	500 (305 with ground truth)

Authors’ Contact Information: Atragada Venkata Naga Sai Ganesh (24110060), atragada.ganesh@iitgn.ac.in, IIT Gandhinagar, Gandhinagar, India; Manemoni Sumanjali (24110195), manemoni.sumanjali@iitgn.ac.in, IIT Gandhinagar, Gandhinagar, India; Sidda Divya (24110337), 24110337@iitgn.ac.in, IIT Gandhinagar, Gandhinagar, India; Divyam Sahu (25120028), divyamsahu17@iitgn.ac.in, IIT Gandhinagar, Gandhinagar, India.

3 Data Preprocessing

First we started with cleaning all the passages by lowercasing, removing punctuation, tokenized using NLTK, removed stop words, and kept only alphabetic tokens.

When we tried to load all 1.99M passages system crashed, hence processed the passages in 4 batches(500k) and saved every batch in google drive as a pickle file for crash recovery.

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r'^\w\s', '', text)
    tokens = word_tokenize(text)
    return [t for t in tokens if t not in STOP_WORDS and t.isalpha()]
```

4 Methodology

We used same preprocessing pipeline for all three designs, and use FAISS IndexFlatIP with L2-normalized embeddings, which gave cosine similarity search.

4.1 Design 1: Word2Vec + Query Expansion

Using Gensim [5] we have trained a Word2Vec skip-gram model on full preprocessed corpus with a vector_size=300, window=5, min_count=2, and 5 epochs. The final vocabulary has 414,512 tokens. The we stored each passage by averaging it's word vectors and store as a float32 np.memmap file so it does not need to get loaded fully into RAM.

We wrote a QueryEncoder class that does query expansion for query encoding, For every query token it finds the top-N Word2Vec neighbours with cosine similarity ≥ 0.5 , and then builds a weighted average: original tokens get weight 1.0 and expanded tokens get 0.7. This way the expansion adds raetable words without losing the original query's meaning.

```
# weight=1.0 for original, weight=0.7 for expanded
vecs += [1.0 * model.wv[t] for t in tokens if t in model.wv]
vecs += [0.7 * model.wv[t] for t in expanded if t in model.wv]
```

4.2 Design 2: SBERT Bi-Encoder

For this we used all-MiniLM-L6-v2 [6], a 6-layer transformer model which encodes the full sentence and understands contextual meaning, unlike Word2Vec which just do averages word vectors. We encoded all the 1.99M passages in batches of 512 into 384-dimensional L2-normalized vectors stored as a memmap file. So, At query time the raw query text will be passed directly to SBERT and searched against the FAISS index.

Table 2. SBERT model properties.

Property	Value
Architecture	6-layer MiniLM Transformer
Embedding dim	384
Batch size	512 passages
Normalization	L2-normalized
Storage	float32 memmap

4.3 Design 3: Hybrid Pipeline

So, here we combine the speed of Word2Vec with the accuracy of SBERT (best of both worlds). Word2Vec will first retrieve the top-300 candidates quickly, then we removed duplicates, then SBERT reranks only those 300 using a temporary per-query FAISS index, and the final top- k will be returned. We set `rerank_k=300` to give the reranker enough diversified data to work with. The trade-off here is that if a relevant passage does not appear in 1st stage from Word2vec search, SBERT never sees it — this is the recall limit of this approach.

```
def search_hybrid(query, k=10, rerank_k=300):
    candidates = search_w2v(query, k=rerank_k) # Stage 1
    reranked = rerank_with_sbert(query, candidates) # Stage 2+3
    return reranked[:k] # Stage 4
```

5 Libraries

Table 3. Libraries used.

Library	Version	Purpose
gensim	4.4.0	Word2Vec training
sentence-transformers	5.4.0	SBERT encoding
faiss-cpu	1.13.2	Vector search
datasets	—	Load MS MARCO
nltk	3.x	Tokenization, stopwords
numpy	2.0.2	Vectors, memmap
gradio	—	Web demo
Google Colab + Drive	—	Runtime & storage

6 Evaluation

We used the MS MARCO `is_selected` flags for ground truth: passages marked 1 for a query are considered relevant. We calculated five standard IR metrics at $k \in \{5, 10\}$:

- **P@k** = $|\text{relevant} \cap \text{top-}k|/k$ — how many returned results are truly relevant.
- **R@k** = $|\text{relevant} \cap \text{top-}k|/|\text{relevant}|$ — how much of the relevant content we managed to find.
- **F1@k** = $2PR/(P + R)$ — combined precision and recall in one number.

- **MRR@k** = $1/\text{rank of first relevant result}$ — rewards systems that put the relevant result at the top.
- **NDCG@k** = DCG/IDCG , where $\text{DCG} = \sum \text{rel}(i)/\log_2(i+1)$ — measures overall ranking quality, giving high credit to relevant results ranked higher.

We evaluated three models on the same 500 queries; 305 had at least one relevant passage and were used for scoring.

7 Results

7.1 Model Comparison

Table 4 show the full results. The gap between Word2Vec and SBERT is very large across all metrics, this is expected considering significant difference in the models.

Table 4. Retrieval metrics averaged over 305 queries. Bold = best.

Model	P@5	R@5	F1@5	MRR@5	NDCG@5	P@10	R@10	MRR@10	NDCG@10
Word2Vec	0.019	0.090	0.031	0.047	0.056	0.012	0.106	0.049	0.061
SBERT	0.125	0.606	0.206	0.386	0.438	0.076	0.736	0.403	0.481
Hybrid	0.065	0.307	0.106	0.226	0.242	0.035	0.330	0.229	0.250

SBERT is almost $6\times$ better than Word2Vec on precision and recall. While The Hybrid performs better over Word2Vec (MRR@5 goes from 0.047 to 0.226) but cannot match standalone SBERT because of the recall ceiling — So documents Word2Vec misses, SBERT never knows to rerank.

	P@5	R@5	F1@5	MRR@5	NDCG@5	P@10	R@10	F1@10	MRR@10	NDCG@10	n_evaluated	n_skipped
Word2Vec	0.0190	0.0896	0.0311	0.0465	0.0555	0.0115	0.1060	0.0206	0.0488	0.0611	305.0	195.0
BERT	0.1246	0.6060	0.2060	0.3860	0.4376	0.0764	0.7361	0.1380	0.4030	0.4807	305.0	195.0
Hybrid	0.0649	0.3071	0.1064	0.2256	0.2422	0.0351	0.3301	0.0631	0.2285	0.2501	305.0	195.0

Fig. 1. Results table across all metrics at $k=5$ and $k=10$.

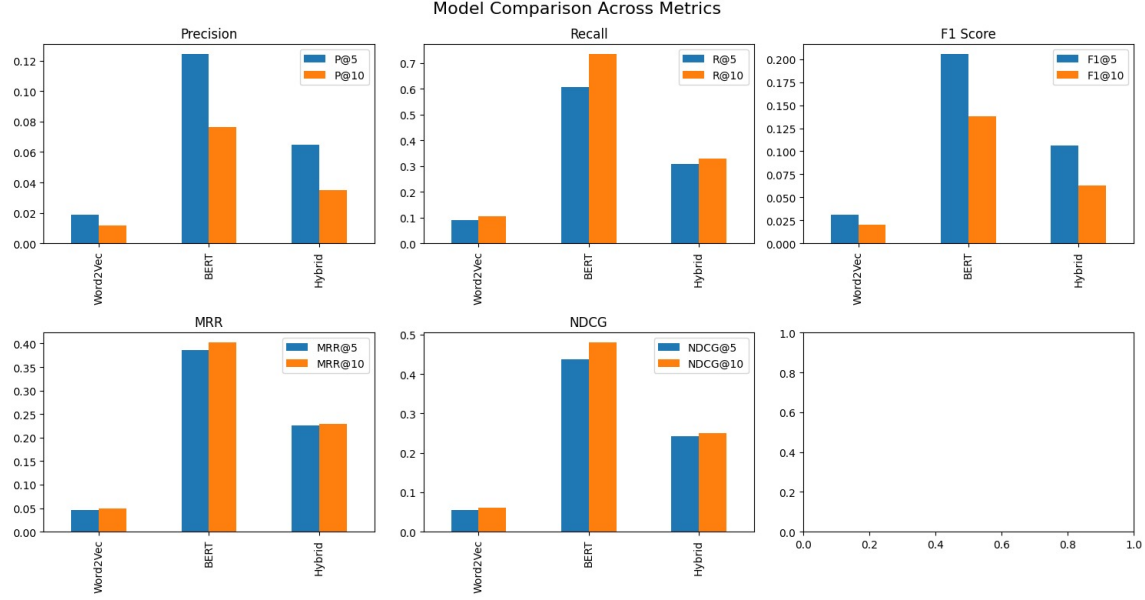


Fig. 2. Bar chart comparison of all five metrics for the three models.

7.2 Query Expansion Ablation

To get the best expansion size, we tested $N \in \{3, 5, 10\}$ neighbours per query token and re-evaluated Word2Vec on 305 queries for each setting.

	P@5	R@5	F1@5	MRR@5	NDCG@5	P@10	R@10	F1@10	MRR@10	NDCG@10	n_evaluated	n_skipped
N=3	0.0223	0.1060	0.0366	0.0629	0.0721	0.0151	0.1421	0.0271	0.0673	0.0840	305.0	195.0
N=5	0.0190	0.0896	0.0311	0.0465	0.0555	0.0115	0.1060	0.0206	0.0488	0.0611	305.0	195.0
N=10	0.0125	0.0601	0.0205	0.0359	0.0407	0.0098	0.0929	0.0177	0.0406	0.0513	305.0	195.0

Fig. 3. Ablation table: metrics for $N=3, 5, 10$.

Query Expansion Experiment

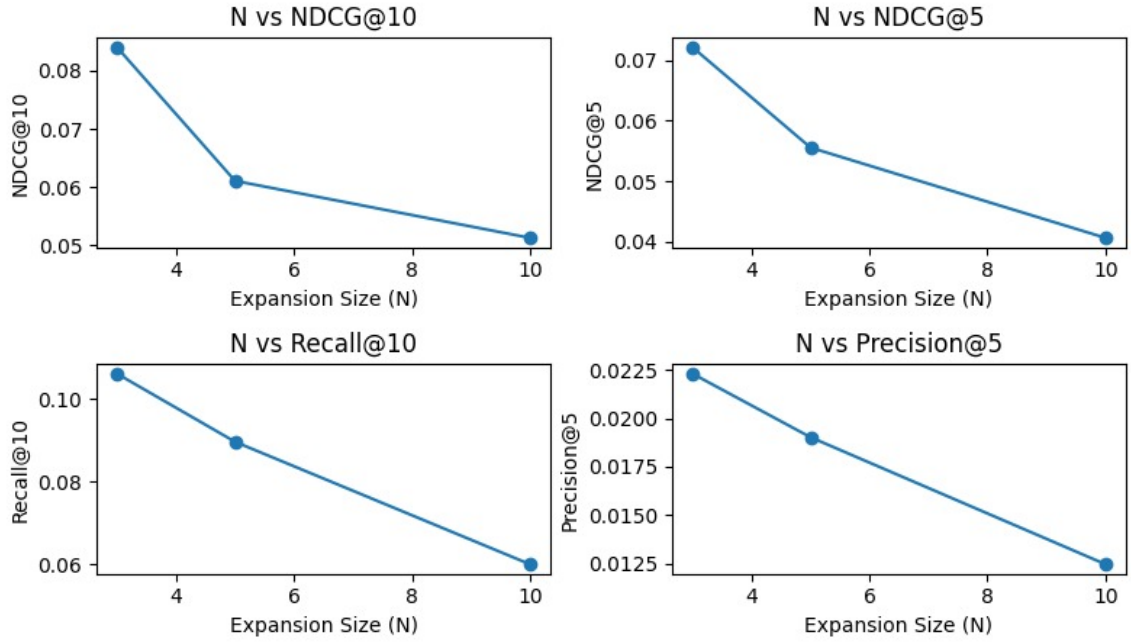


Fig. 4. Line plots of NDCG and Recall vs expansion size N .

Table 5. Query expansion ablation (305 queries). Bold = best.

N	P@5	R@5	MRR@5	NDCG@5	NDCG@10
3	0.022	0.106	0.063	0.072	0.084
5	0.019	0.090	0.047	0.056	0.061
10	0.013	0.060	0.036	0.041	0.051

Results show $N=3$ gave the best results across all metrics. As N increases, performance keeps decreasing — at $N=10$, P@5 falls by 44% compared to $N=3$. The reason is, larger N brings in loosely related words which dilute the query vector and fetch in irrelevant passages. Concludes that too much expansion is not always better.

7.3 Query Latency

```
query="what are the best iits in india?"
speed_df = compare_single_random_query(query, k=5)
speed_df
```

	Method	Query	Time_ms
0	Word2Vec	what are the best iits in india?	933.285715
1	Hybrid	what are the best iits in india?	1146.743541
2	SBERT	what are the best iits in india?	1622.997113

Fig. 5. Query latency for each method at $k=5$.

Table 6. Latency comparison (query: “what are the best IITs in India?”, $k=5$).

Method	Latency	Relative
Word2Vec	933 ms	1.00×
Hybrid	1147 ms	1.23×
SBERT	1623 ms	1.74×

Word2Vec is the fastest because there is no neural model executing at query time — just a simple vector lookup and FAISS search. SBERT runs a full transformer forward pass per query, that’s why it is slowest. The Hybrid is in between since it runs SBERT only on first 300 candidates instead of complete 1.99M passages.

8 Web Application

We also built an interactive Gradio app by using that users can type any query, choose which design to use, and also can adjust the top- k value. Each result shows the rank, cosine similarity score, full passage text, and how long(latency) the query took. The screenshots below are for the query “*what are the energies required for photosynthesis process?*” on all three designs.

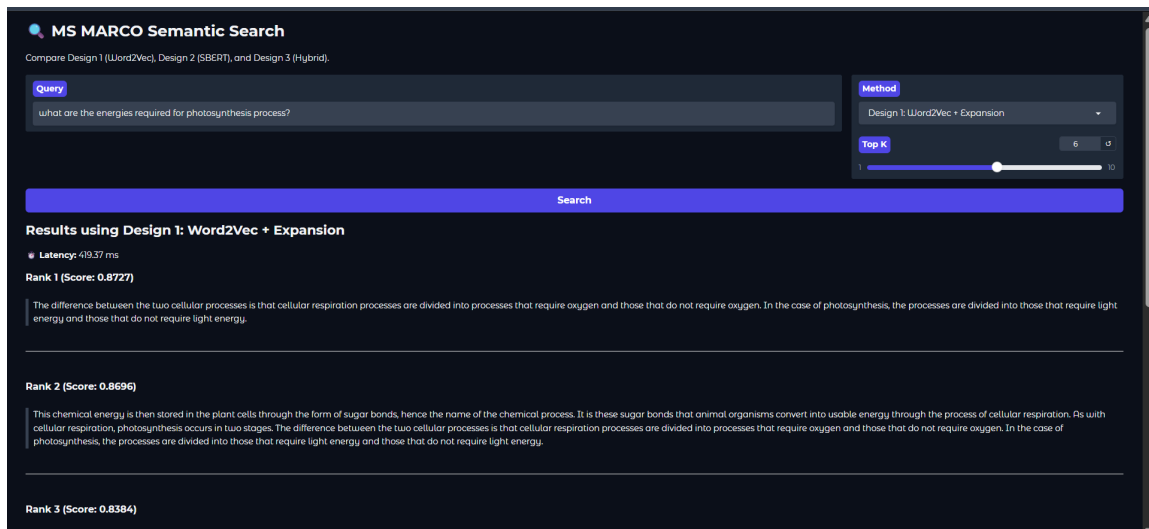


Fig. 6. Design 1 (Word2Vec + Expansion), latency: 419 ms.

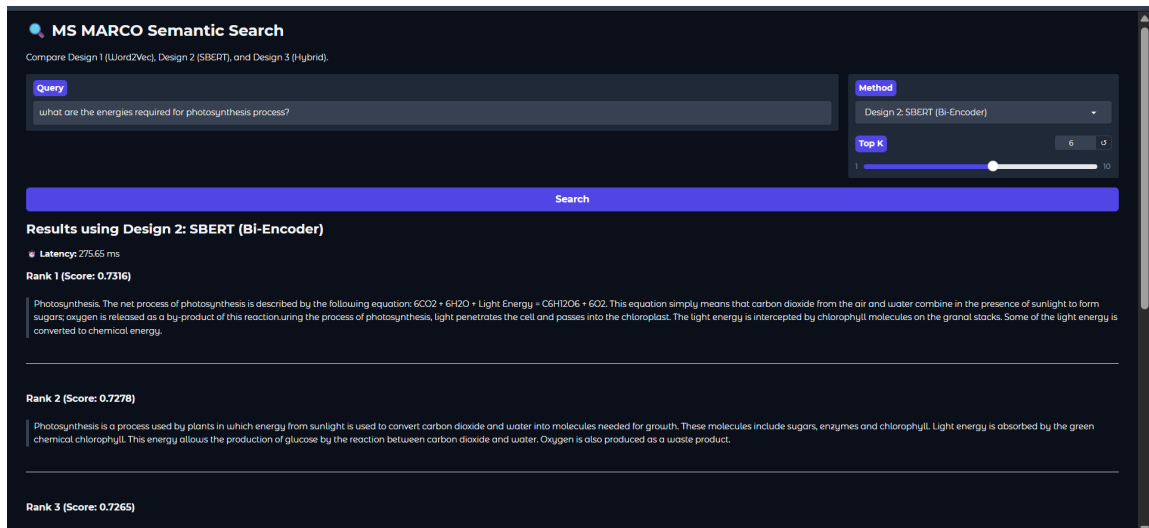


Fig. 7. Design 2 (SBERT), latency: 275 ms.

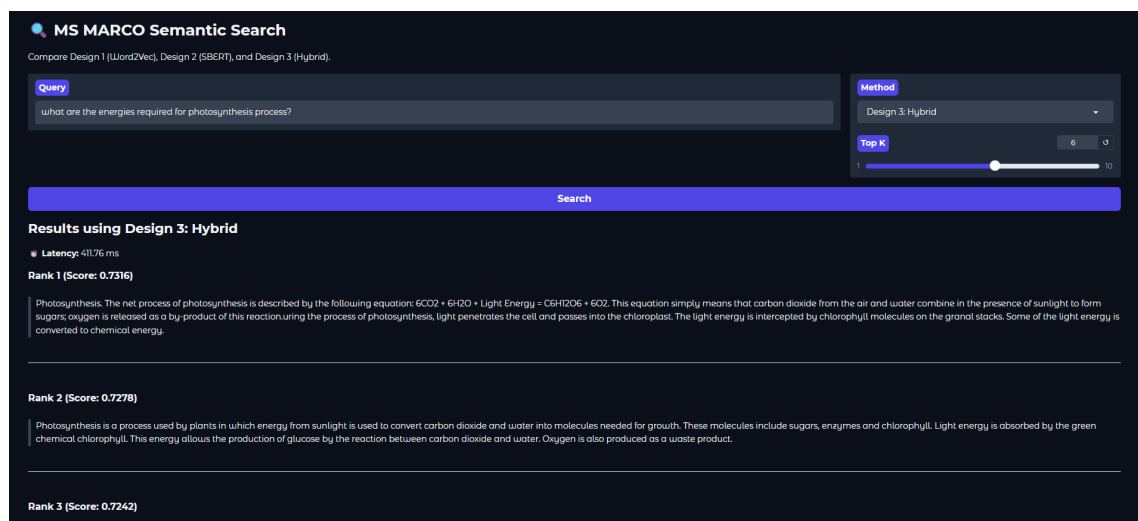


Fig. 8. Design 3 (Hybrid), latency: 411 ms.

We noticed that SBERT and Hybrid both return passages that directly explain photosynthesis and the energy equation. Word2Vec on the other hand returns passages about cellular respiration that just happen to mention photosynthesis somewhere, which shows why the metric numbers are so different.

9 Discussion

Why SBERT wins. Word2Vec just give each word one fixed vector, so it fails to handle words with multiple meanings, and averaging vectors loses word order completely. SBERT processes the full sentence through attention layers and was fine-tuned on semantic similarity tasks, so it already do know that “car” and “vehicle” are the same thing and also understands phrases like “global warming” in context. Word2Vec has to learn all of this from co-occurrence counts alone, which is not sufficient.

Hybrid ceiling. Using a quick retriever followed by an expensive accurate reranker is a well-known pattern in search systems. The problem is that Hybrid quality is completely cieled by what Word2Vec finds in Stage 1. Using somewhat better first-stage retriever would actually improve the Hybrid results.

Expansion size. $N=3$ adds only the most closely related words, so the query vector stays close to the original meaning. At $N=10$, too many far fetched related words are added and the query vector goes towards an average of unrelated concepts.

10 Limitations

- **Word2Vec:** one fixed vector per word means it cannot handle polysemy, and vocabulary is limited to what has seen in training.
- **SBERT:** the model we used was not actually fine-tuned specifically on MS MARCO, so a domain-specific model would likely do better. Also, IndexFlatIP do brute-force $O(N)$ search and will not actually scale to very large corpora.

- **Hybrid:** the Stage 1 recall ceiling is a hard limit — relevant passages not in the top-300 from Word2Vec are gone.
- **Evaluation:** 195 queries had no ground truth and were skipped, which might introduce some bias. Ground truth is binary, latency was measured on a single query run on Colab CPU, and we only evaluated on 500 queries from 200K.

11 Conclusion

From our experiments, SBERT is the best design with almost 6–7× higher precision and recall than Word2Vec. This does confirm that contextual sentence embeddings are much better for passage retrieval than static word averaging. Although Word2Vec is still useful when speed is main requirement than accuracy. The Hybrid comes in the middle but its quality is limited by how the first stage is. For query expansion, $N=3$ works best — going more just adds noise. FAISS made exact cosine search over 1.99M passages practical even on CPU, which was a key finding for us.

References

- [1] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2017. FAISS – Facebook AI Similarity Search. <https://faiss.ai/>
- [2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press. <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-information-retrieval-1.html>
- [3] Microsoft. 2016. MS MARCO Dataset. <https://microsoft.github.io/msmarco/>
- [4] Microsoft. 2023. MS MARCO on Hugging Face. https://huggingface.co/datasets/microsoft/ms_marco
- [5] Radim Řehůřek and Petr Sojka. 2010. Gensim Word2Vec Documentation. <https://radimrehurek.com/gensim/models/word2vec.html>
- [6] Nils Reimers and Iryna Gurevych. 2019. Sentence Transformers Documentation. <https://www.sbert.net/>
- [7] Towards Data Science. 2019. Word2Vec Explained. <https://towardsdatascience.com/word2vec-explained-49c52b4ccb71>